

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XII
SOLID Prinsiples



Disusun Oleh:
Alwisyah Putra 105224052

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

DAFTAR ISI

I. PENDAHULUAN.....	2
III. VARIABEL.....	3
1. KrsPrinter.java.....	3
2. ValidatorMatKul.java.....	3
3. MataKuliah.java.....	3
4. MatkulPraktikum.java.....	3
5. Praktikum.java.....	3
6. BidikMisi.java.....	3
7. CalculateUKT.java.....	3
8. DataUKT.java.....	3
9. Internasional.java.....	3
10. Regular.java.....	3
11. UktCalculateStrategy.java.....	3
12. MySQLDatabaseConnection.....	3
13. App.java.....	3
IV. CONSTRUCTOR & METHOD.....	3
1. KrsPrinter.java.....	3
2. ValidatorMatKul.java.....	3
3. MataKuliah.java.....	3
4. MatkulPraktikum.java.....	3
5. Praktikum.java.....	3
6. BidikMisi.java.....	3
7. CalculateUKT.java.....	3
8. DataUKT.java.....	3
9. Internasional.java.....	3
10. Regular.java.....	3
11. UktCalculateStrategy.java.....	3
12. MySQLDatabaseConnection.....	3
13. App.java.....	3
V. DOKUMENTASI DAN PEMBAHASAN CODE.....	3
VI. KESIMPULAN.....	10
VII. DAFTAR PUSTAKA.....	10

I. PENDAHULUAN

Sistem Kartu Rencana Studi (KRS) di kampus mengalami krisis arsitektur yang parah. Selama masa pengisian KRS, ribuan mahasiswa mengalami kelambatan hingga crash karena seluruh beban logika aplikasi terpusat pada satu kelas raksasa bernama SistemKRSManager. Kelas ini menangani segalanya mulai dari validasi prasyarat, perhitungan UKT, pencetakan PDF, hingga koneksi database yang di-hardcode langsung dengan instansiasi MySQLDatabaseConnection. Ketika programmer mencoba mengubah format kop surat PDF, perubahan kecil itu tanpa sengaja merusak algoritma penyimpanan data sehingga ribuan data KRS gagal tersimpan. Masalah semakin memburuk saat kampus menambah jenis mata kuliah baru seperti Teori, Praktikum, dan Kuliah Lapangan/KKN. Programmer terpaksa memaksa semua jenis kelas mengimplementasikan metode alokasiAsistenLab() dan cekPeralatanPraktikum(), padahal mata kuliah Teori dan KKN tidak memerlukannya, sehingga harus melempar UnsupportedOperationException yang sering memicu error saat iterasi data. Di sisi lain, perhitungan UKT untuk jalur Regular, Karyawan, Internasional, dan Bidikmisi dipenuhi oleh ratusan baris if-else yang bertumpuk, membuat setiap penambahan jalur baru harus membongkar dan merombak seluruh sistem. Kini kampus menuntut migrasi dari MySQL ke Cloud NoSQL dan penambahan skema SKS baru Merdeka Belajar Kampus Merdeka (MBKM), namun arsitektur yang kaku dan rapuh membuat sistem tidak siap menghadapi perubahan apapun. Tugas ini menantang untuk menganalisis penyakit arsitektur yang menjangkiti sistem tersebut dan merancang ulang struktur kodenya agar tangkas, tahan terhadap perubahan, serta mampu mengakomodasi migrasi database dan penambahan jalur mahasiswa baru secara elegan.

II. HASIL ANALISIS

1. Single Responsibility Principle (SRP)

Kelas SistemKRSManager menangani terlalu banyak tanggung jawab sekaligus: validasi prasyarat mata kuliah, perhitungan UKT, pencetakan KRS dalam format PDF, hingga penyimpanan riwayat pengambilan kelas dengan koneksi database MySQL yang di-hardcode. Akibatnya, perubahan kecil pada format kop surat

PDF tanpa sengaja merusak algoritma penyimpanan data dan menyebabkan ribuan data KRS mahasiswa gagal tersimpan. Ketika satu kelas memiliki banyak alasan untuk berubah, risiko bug meningkat drastis karena modifikasi pada satu fitur dapat berdampak tidak terduga pada fitur lainnya. Seharusnya setiap kelas hanya memiliki satu tugas spesifik, misalnya pemisahan antara KrsPrinter untuk pencetakan, ValidatorMatKul untuk validasi, dan CalculateUKT untuk perhitungan biaya.

2. Open/Closed Principle (OCP)

Sistem tidak terbuka untuk ekstensi namun terpaksa dimodifikasi setiap kali ada penambahan fitur baru. Saat kampus memperkenalkan jenis mata kuliah baru seperti Teori, Praktikum, dan Kuliah Lapangan/KKN, programmer harus merombak antarmuka induk OperasiMataKuliah yang mewajibkan semua jenis kelas mengimplementasikan metode alokasiAsistenLab() dan cekPeralatanPraktikum(). Akibatnya, mata kuliah Teori dan KKN yang sebenarnya tidak memerlukan asisten lab maupun peralatan praktikum terpaksa mengimplementasikan metode tersebut dengan melempar UnsupportedOperationException, yang justru menjadi sumber error mendadak saat sistem melakukan iterasi data. Demikian pula, setiap penambahan jalur mahasiswa baru seperti Merdeka Belajar Kampus Merdeka (MBKM) mengharuskan pengembang membongkar dan merombak ratusan baris if-else perhitungan UKT yang bertumpuk di dalam SistemKRSManger, yang kerap kali merusak rumus perhitungan untuk mahasiswa lama. Seharusnya sistem dirancang agar fitur baru dapat ditambahkan tanpa mengubah kode yang sudah ada, misalnya dengan memisahkan perilaku praktikum ke antarmuka tersendiri dan menerapkan Strategy Pattern untuk perhitungan UKT.

3. Liskov Substitution Principle (LSP)

Substitusi objek anak ke posisi induk tidak berjalan dengan aman karena adanya UnsupportedOperationException. Ketika semua jenis mata kuliah dipaksa mengimplementasikan OperasiMataKuliah, mata kuliah Teori dan KKN yang merupakan subclass secara konsep harusnya bisa menggantikan induknya tanpa masalah, namun kenyataannya mereka melempar exception saat metode praktikum dipanggil. Ini memecah kontrak bahwa subclass harus bisa digunakan

sebagai pengganti superclass tanpa mengubah kebenaran program. Error mendadak yang sering muncul saat iterasi data terjadi karena sistem mengasumsikan semua objek mata kuliah mendukung operasi praktikum, padahal sebagian besar tidak. Seharusnya antarmuka dipisah lebih spesifik, seperti MataKuliah untuk data dasar dan Praktikum khusus untuk mata kuliah yang memang memiliki kegiatan praktikum, sehingga hanya objek yang relevan yang mengimplementasikan perilaku tersebut.

4. Interface Segregation Principle (ISP)

Antarmuka OperasiMataKuliah terlalu besar dan memaksakan metode yang tidak relevan bagi sebagian implementasinya. Interface tersebut mewajibkan semua jenis kelas mengimplementasikan alokasiAsistenLab() dan cekPeralatanPraktikum(), padahal metode tersebut hanya dibutuhkan oleh mata kuliah praktikum saja. Mata kuliah Teori dan Kuliah Lapangan/KKN menjadi korban pemaksaan ini dan harus menyediakan implementasi bohongan yang melempar exception. Hal ini menunjukkan bahwa antarmuka seharusnya dipecah menjadi bagian-bagian yang lebih kecil dan spesifik, sehingga class hanya mengimplementasikan method yang benar-benar dibutuhkannya. Dengan pemisahan antarmuka, mata kuliah praktikum bisa mengimplementasikan Praktikum secara terpisah dari MataKuliah, sementara mata kuliah teori cukup mengimplementasikan MataKuliah tanpa beban method yang tidak relevan.

5. Dependency Inversion Principle (DIP)

Modul tingkat tinggi bergantung langsung pada modul tingkat rendah secara konkret, bukan pada abstraksi. SistemKRSManager menginstansiasi MySQLDatabaseConnection secara langsung dengan hardcode, sehingga ketika kampus ingin bermigrasi dari MySQL ke infrastruktur Cloud NoSQL, seluruh sistem harus dibongkar karena ketergantungan yang kaku tersebut. Seharusnya sistem bergantung pada antarmuka abstrak seperti DatabaseConnection, sehingga implementasi konkret MySQL atau Cloud NoSQL bisa diganti tanpa mengubah kode bisnis utama. Begitu pula pada perhitungan UKT, ratusan baris if-else yang menangani jalur Regular, Karyawan, Internasional, dan Bidikmisi secara langsung di dalam satu kelas membuat logika bisnis terkait erat dengan detail implementasi masing-masing jalur. Seharusnya digunakan Strategy Pattern

dengan antarmuka `UktCalculateStrategy`, sehingga `CalculateUKT` hanya bergantung pada abstraksi strategi dan setiap jalur mahasiswa bisa ditambahkan sebagai implementasi konkret yang baru tanpa mengubah kode yang sudah ada.

III. VARIABEL

1. `KrsPrinter.java`

Nama Variabel	Tipe Data	Fungsi
<code>daftarMataKuliah</code>	<code>List<MataKuliah></code>	Parameter untuk menyimpan daftar mata kuliah yang akan dicetak dalam KRS
<code>namaMahasiswa</code>	<code>String</code>	Parameter untuk menyimpan nama mahasiswa yang akan dicetak KRS-nya
<code>sb</code>	<code>StringBuilder</code>	Variabel lokal untuk membangun string daftar mata kuliah
<code>totalSKS</code>	<code>int</code>	Variabel lokal untuk menghitung total SKS dari semua mata kuliah
<code>mk</code>	<code>MataKuliah</code>	Variabel iterasi dalam loop for-each untuk mengakses setiap mata kuliah

2. `ValidatorMatKul.java`

Nama Variabel	Tipe Data	Fungsi
<code>MATA_KULIAH_TANPA_PRASYARAT</code>	<code>List<String></code>	Variabel static final yang berisi daftar mata kuliah yang tidak memiliki prasyarat

3. `MataKuliah.java`

Nama Variabel	Tipe Data	Fungsi
-	-	-

4. `MatkulPraktikum.java`

Nama Variabel	Tipe Data	Fungsi
---------------	-----------	--------

namaMatkul	String	Menyimpan nama mata kuliah praktikum
sks	int	Menyimpan jumlah SKS mata kuliah praktikum

5. Praktikum.java

Nama Variabel	Tipe Data	Fungsi
-	-	-

6. BidikMisi.java

Nama Variabel	Tipe Data	Fungsi
UKT_BIDIK_MISI	double	Konstanta untuk nominal UKT program Bidik Misi (0, gratis)

7. CalculateUKT.java

Nama Variabel	Tipe Data	Fungsi
strategy	UktCalculateStrategy	Menyimpan strategi perhitungan UKT yang akan digunakan

8. DataUKT.java

Nama Variabel	Tipe Data	Fungsi
jurusan	String	Menyimpan nama jurusan mahasiswa
tingkat	String	Menyimpan tingkat/semester mahasiswa
prestasi	String	Menyimpan status prestasi mahasiswa (berprestasi/tidak)

9. Internasional.java

Nama Variabel	Tipe Data	Fungsi
UKT_INTERNASIONAL	double	Konstanta untuk nominal UKT mahasiswa internasional

		(150.000.000)
--	--	---------------

10. Regular.java

Nama Variabel	Tipe Data	Fungsi
UKT_DASAR	double	Konstanta untuk nominal UKT dasar mahasiswa regular (50.000.000)
DISKON_BERPRESTASI	double	Konstanta untuk diskon UKT jika berprestasi (20.000.000)
STATUS_BERPRESTASI	String	Konstanta string untuk status berprestasi

11. UktCalculateStrategy.java

Nama Variabel	Tipe Data	Fungsi
-	-	-

12. MySQLDatabaseConnection

Nama Variabel	Tipe Data	Fungsi
-	-	-

13. App.java

IV. CONSTRUCTOR & METHOD

1. KrsPrinter.java

Nama Method	Jenis	Fungsi
printPDF	Instance	Mencetak KRS

2. ValidatorMatKul.java

Nama Method	Jenis	Fungsi
ValidatorMatKul()	Default	Membuat instance ValidatorMatKul (constructor default implisit)
validasiPrasyarat	instance	Memvalidasi apakah prasyarat mata kuliah terpenuhi untuk mahasiswa tertentu

3. MataKuliah.java

Nama Method	Jenis	Fungsi
getNamaMatkul	instance (abstract)	Mengembalikan nama mata kuliah
getSKS	instance (abstract)	Mengembalikan jumlah SKS mata kuliah

4. MatkulPraktikum.java

Nama Method	Jenis	Fungsi
MatkulPraktikum(String namaMatkul, int sks)	Constructor	Membuat instance MatkulPraktikum dengan nama dan SKS tertentu
getNamaMatkul	instance	Mengembalikan nama mata kuliah praktikum (override dari MataKuliah)
getSKS	instance	Mengembalikan jumlah SKS mata kuliah praktikum (override dari MataKuliah)
alokasiAsistenLab	instance	Mengalokasikan asisten lab untuk praktikum (implementasi dari Praktikum)
cekPeralatanPraktikum	instance	Memeriksa ketersediaan peralatan praktikum (implementasi dari Praktikum)

5. Praktikum.java

Nama Method	Jenis	Fungsi
alokasiAsistenLab	instance (abstract)	Mengalokasikan asisten lab untuk praktikum
cekPeralatanPraktikum	instance (abstract)	Memeriksa ketersediaan peralatan praktikum

6. BidikMisi.java

Nama Method	Jenis	Fungsi
BidikMisi()	Default	Membuat instance strategi BidikMisi
calculateUkt	instance	Menghitung UKT untuk program

		Bidik Misi (gratis, override dari UktCalculateStrategy)
--	--	---

7. CalculateUKT.java

Nama Method	Jenis	Fungsi
CalculateUKT(UktCalculateStrategy strategy)	Constructor	Membuat instance CalculateUKT dengan strategi tertentu
setStrategy	instance	Mengubah strategi perhitungan UKT yang digunakan
calculate	instance	Menghitung UKT berdasarkan strategi yang telah diatur

8. DataUKT.java

Nama Method	Jenis	Fungsi
DataUKT(String jurusan, String tingkat, String prestasi)	Parameterized	Membuat instance DataUKT dengan data jurusan, tingkat, dan prestasi
getJurusan	instance	Mengembalikan nama jurusan
getTingkat	instance	Mengembalikan tingkat/semester
getPrestasi	instance	Mengembalikan status prestasi

9. Internasional.java

Nama Method	Jenis	Fungsi
Internasional()	Default	Membuat instance strategi Internasional
calculateUkt	instance	Menghitung UKT untuk mahasiswa internasional (override dari UktCalculateStrategy)

10. Regular.java

Nama Method	Jenis	Fungsi
Regular()	Default	Membuat instance strategi Regular
calculateUkt	instance	Menghitung UKT untuk mahasiswa regular dengan diskon berprestasi (override dari UktCalculateStrategy)

11. Karyawan

Nama Method	Jenis	Fungsi
Karyawan()	Default	Membuat instance strategi Karyawan
calculateUkt	instance	Menghitung UKT untuk mahasiswa karyawan (override dari UktCalculateStrategy)

12. UktCalculateStrategy.java

Nama Method	Jenis	Fungsi
calculateUkt	instance (abstract)	Method abstrak untuk menghitung UKT berdasarkan data mahasiswa

13. MySQLDatabaseConnection

Nama Method	Jenis	Fungsi
connect()	instance (abstract)	Membuat koneksi ke database MySQL
disconnect()	instance (abstract)	Memutuskan koneksi dari database MySQL
isConnected()	instance (abstract)	Mengecek apakah koneksi ke database masih aktif, mengembalikan boolean
executeQuery(String query)	instance (abstract)	Menjalankan query SQL yang diberikan sebagai parameter

14. App.java

Nama Method	Jenis	Fungsi

V. DOKUMENTASI DAN PEMBAHASAN CODE

1. KrsPrinter.java

```

1  import java.util.List;
2
3  public class KrsPrinter {
4
5      public void printPDF(String namaMahasiswa, List<MataKuliah> daftarMataKuliah) {
6          System.out.println("===KOP SURAT UNIVERSITAS===");
7          System.out.println("Nama Mahasiswa: " + namaMahasiswa);
8
9
10         StringBuilder sb = new StringBuilder();
11         int totalSKS = 0;
12         for (MataKuliah mk : daftarMataKuliah) {
13             sb.append(mk.getNamaMatkul()).append(" ").append(mk.getSKS()).append(" SKS), ");
14             totalSKS += mk.getSKS();
15         }
16
17         System.out.println("Daftar Mata Kuliah: " + sb.toString());
18         System.out.println("Total SKS: " + totalSKS);
19         System.out.println("=====");
20         System.out.println("KRS PDF berhasil dibuat untuk " + namaMahasiswa);
21     }
22 }

```

KrsPrinter.java bertugas mencetak Kartu Rencana Studi dalam format teks sederhana yang mensimulasikan dokumen PDF. Method printPDF menerima parameter nama mahasiswa beserta daftar mata kuliah dalam bentuk List, kemudian mengiterasi setiap mata kuliah menggunakan perulangan for-each untuk mengumpulkan nama dan SKS ke dalam StringBuilder sekaligus menjumlahkan total SKS. Hasilnya ditampilkan dengan format kop surat universitas, identitas mahasiswa, daftar mata kuliah yang dipisahkan koma, total SKS, dan konfirmasi bahwa KRS berhasil dibuat. Prinsip SOLID yang diterapkan: Class ini menerapkan Single Responsibility Principle (SRP) karena hanya bertanggung jawab untuk mencetak KRS. Class ini juga menerapkan Dependency Inversion Principle (DIP) karena bergantung pada abstraksi List<MataKuliah>

2. ValidatorMatKul.java

```

1  import java.util.Arrays;
2  import java.util.List;
3
4  public class ValidatorMatKul {
5
6
7      private static final List<String> MATA_KULIAH_TANPA_PRASYARAT = Arrays.asList(
8          "PBO", "Basis Data", "Algoritma");
9
10     public boolean validasiPrasyarat(String mahasiswaId, String kodeMatkul) {
11         System.out.println("Memeriksa prasyarat untuk " + kodeMatkul + "...");
12
13
14         if (MATA_KULIAH_TANPA_PRASYARAT.contains(kodeMatkul)) {
15             System.out.println( kodeMatkul + " tidak memiliki prasyarat");
16             return true;
17         }
18
19
20         System.out.println(" Prasyarat " + kodeMatkul + " terpenuhi untuk mahasiswa " + mahasiswaId);
21         return true;
22     }
23 }

```

ValidatorMatKul.java berfungsi sebagai mesin validasi prasyarat pendaftaran mata kuliah yang menyimpan daftar mata kuliah yang tidak memerlukan prasyarat dalam sebuah List statis berisi "PBO", "Basis Data", dan "Algoritma". Method validasiPrasyarat menerima parameter mahasiswaId dan kodeMatkul, kemudian memeriksa apakah kode mata kuliah tersebut terdapat dalam daftar bebas prasyarat, jika ya maka langsung mengembalikan true dengan mencetak informasi bahwa mata kuliah tidak memiliki prasyarat, namun dalam implementasi saat ini semua mata kuliah termasuk yang tidak ada dalam daftar tetap mengembalikan true sehingga validasi selalu berhasil. Prinsip SOLID yang diterapkan: Class ini menerapkan Single Responsibility Principle (SRP) karena hanya memiliki satu alasan untuk berubah, yaitu perubahan aturan validasi prasyarat. Namun, class ini memiliki kelemahan dalam Open/Closed Principle (OCP) karena daftar mata kuliah tanpa prasyarat masih hardcoded, sehingga perlu modifikasi langsung untuk menambah mata kuliah baru.

3. MataKuliah.java



```
1  public interface MataKuliah {  
2      String getNamaMatkul();  
3      int getSKS();  
4  }  
5
```

interface yang mendefinisikan kontrak dasar bagi setiap entitas mata kuliah dalam sistem, di mana setiap class yang mengimplementasikannya wajib menyediakan method `getNamaMatkul` untuk mengembalikan nama mata kuliah dan method `getSKS` untuk mengembalikan jumlah satuan kredit semester, sehingga memastikan konsistensi struktur data mata kuliah di seluruh program. Prinsip SOLID yang diterapkan: Interface ini merupakan penerapan Interface Segregation Principle (ISP) karena hanya berisi method-method yang esensial untuk semua jenis mata kuliah, tidak memaksakan class untuk mengimplementasikan method yang tidak relevan. Interface ini juga mendukung Liskov Substitution Principle (LSP) karena setiap implementasi pengganti (`MatkulPraktikum` atau anonymous class) dapat digunakan di mana saja `MataKuliah` diharapkan.

4. `MatkulPraktikum.java`

```

1  public class MatkulPraktikum implements MataKuliah, Praktikum {
2      private String namaMatkul;
3      private int sks;
4      public MatkulPraktikum(String namaMatkul, int sks) {
5          this.namaMatkul = namaMatkul;
6          this.sks = sks;
7      }
8
9      @Override
10     public String getNamaMatkul() {
11         return namaMatkul;
12     }
13
14     @Override
15     public int getSKS() {
16         return sks;
17     }
18
19     @Override
20     public void alokasiAsistenLab() {
21         System.out.println("Asisten lab untuk " + namaMatkul + " telah dialokasikan.");
22     }
23
24     @Override
25     public void cekPeralatanPraktikum() {
26         System.out.println("Peralatan praktikum untuk " + namaMatkul + " telah diperiksa.");
27     }
28
29 }

```

MatkulPraktikum.java mengimplementasikan dua sekaligus yaitu interface MataKuliah dan Praktikum, sehingga class ini merepresentasikan mata kuliah lengkap yang memiliki atribut nama dan SKS sekaligus kemampuan mengelola aspek praktikum. Ketika diinstansiasi, constructor menerima nama mata kuliah dan jumlah SKS yang disimpan sebagai atribut private, kemudian method getNamaMatkul dan getSKS mengembalikan nilai tersebut, sementara method alokasiAsistenLab akan mencetak konfirmasi alokasi asisten dan method cekPeralatanPraktikum akan mencetak konfirmasi pengecekan peralatan ke console. Prinsip SOLID yang diterapkan: Class ini menerapkan Single Responsibility Principle (SRP) karena hanya bertanggung jawab untuk merepresentasikan mata kuliah praktikum dengan atribut dan behavior yang relevan. Class ini juga menerapkan Liskov Substitution Principle (LSP) karena dapat menggantikan MataKuliah di mana pun diperlukan tanpa mengubah kebenaran program, serta menggantikan Praktikum sesuai kontraknya.

5. Praktikum.java



```
1 public interface Praktikum {  
2     void alokasiAsistenLab();  
3     void cekPeralatanPraktikum();  
4 }  
5
```

interface khusus yang menambahkan kontrak tambahan untuk mata kuliah yang memiliki komponen praktikum, yaitu method `alokasiAsistenLab` untuk mengatur penugasan asisten laboratorium dan method `cekPeralatanPraktikum` untuk memverifikasi ketersediaan peralatan praktikum, sehingga memisahkan concern antara mata kuliah teori dan mata kuliah praktikum. Prinsip SOLID yang diterapkan: Interface ini murni menerapkan Interface Segregation Principle (ISP) dengan memisahkan interface khusus praktikum dari interface umum `MataKuliah`, sehingga mata kuliah teori tidak terbebani untuk mengimplementasikan method praktikum yang tidak mereka butuhkan.

6. BidikMisi.java



```
1 public class BidikMisi implements UktCalculateStrategy {  
2     private static final double UKT_BIDIK_MISI = 0;  
3  
4     @Override  
5     public double calculateUkt(DataUKT data) {  
6         return UKT_BIDIK_MISI;  
7     }  
8 }  
9
```

`BidikMisi.java` mengimplementasikan program beasiswa pemerintah dengan menetapkan UKT sebesar nol rupiah. Method `calculateUkt` selalu mengembalikan nilai konstan nol regardless of input data, sehingga mahasiswa penerima beasiswa

Bidik Misi tidak dikenakan biaya kuliah sama sekali. Prinsip SOLID yang diterapkan: Class ini menerapkan Single Responsibility Principle (SRP) dengan hanya menangani perhitungan UKT Bidik Misi. Class ini juga menerapkan Liskov Substitution Principle (LSP) karena dapat menggantikan UktCalculateStrategy tanpa mengubah perilaku yang diharapkan, dan Open/Closed Principle (OCP) sebagai contoh ekstensi strategy baru yang tidak memerlukan perubahan pada CalculateUKT.

7. CalculateUKT.java

```
1  public class CalculateUKT {
2      private UktCalculateStrategy strategy;
3
4      public CalculateUKT() {
5      }
6
7      public CalculateUKT(UktCalculateStrategy strategy) {
8          this.strategy = strategy;
9      }
10
11     public void setStrategy(UktCalculateStrategy strategy) {
12         this.strategy = strategy;
13     }
14
15     public double calculate(DataUKT data) {
16         if (strategy == null) {
17             throw new IllegalStateException("Strategy belum diatur");
18         }
19         return strategy.calculateUkt(data);
20     }
21 }
```

CalculateUKT.java merupakan context class dalam implementasi pattern Strategy yang menyimpan reference ke objek UktCalculateStrategy. Class ini menyediakan constructor default dan constructor dengan parameter strategy, serta method setStrategy untuk mengubah strategy yang aktif secara dinamis. Method calculate memeriksa terlebih dahulu apakah strategy sudah diatur, jika belum maka melemparkan IllegalStateException dengan pesan error, namun jika sudah ada maka mendelegasikan perhitungan ke method calculateUkt milik strategy yang sedang aktif. Prinsip SOLID yang diterapkan: Class ini menerapkan Open/Closed Principle (OCP) karena terbuka untuk ekstensi dengan menambah strategy baru

namun tertutup untuk modifikasi. Class ini juga menerapkan Dependency Inversion Principle (DIP) karena modul high-level (CalculateUKT) bergantung pada abstraksi UktCalculateStrategy, bukan pada detail implementasi konkret seperti Regular, Karyawan, atau Internasional.

8. DataUKT.java

```
1  public class DataUKT {
2      private String jurusan;
3      private String tingkat;
4      private String prestasi;
5
6      public DataUKT(String jurusan, String tingkat, String prestasi) {
7          this.jurusan = jurusan;
8          this.tingkat = tingkat;
9          this.prestasi = prestasi;
10     }
11
12     public String getJurusan() {
13         return jurusan;
14     }
15
16     public String getTingkat() {
17         return tingkat;
18     }
19
20     public String getPrestasi() {
21         return prestasi;
22     }
23 }
```

DataUKT.java berperan sebagai data transfer object yang membungkus informasi mahasiswa yang diperlukan untuk perhitungan UKT, yaitu jurusan sebagai String, tingkat atau semester sebagai String, dan status prestasi sebagai String. Constructor menerima ketiga parameter tersebut dan menyimpannya sebagai atribut private, sementara tiga method getter menyediakan akses terhadap data tersebut untuk digunakan oleh berbagai strategy perhitungan. Prinsip SOLID yang diterapkan: Class ini menerapkan Single Responsibility Principle (SRP) sebagai pure data class yang hanya menyimpan dan menyediakan data. Class ini juga tidak memiliki behavior bisnis, sehingga memisahkan data dari logika perhitungan sesuai prinsip pemisahan concern.

9. Internasional.java

```

1  public class Internasional implements UktCalculateStrategy {
2
3      private static final double UKT_INTERNASIONAL = 150000000;
4
5      @Override
6      public double calculateUkt(DataUKT data) {
7          return UKT_INTERNASIONAL;
8      }
9  }
10

```

Internasional.java menetapkan biaya UKT tertinggi untuk mahasiswa internasional dengan nilai konstan Rp 150.000.000. Sama seperti Karyawan, method calculateUkt tidak memproses data mahasiswa yang diterima melalui parameter dan langsung mengembalikan nilai tetap, mencerminkan kebijakan tarif khusus yang tidak dipengaruhi oleh variabel individual mahasiswa. Prinsip SOLID yang diterapkan: Class ini menerapkan Single Responsibility Principle (SRP) dengan fokus pada satu kategori perhitungan. Class ini juga menerapkan Liskov Substitution Principle (LSP) sebagai implementasi yang dapat menggantikan parent interface, dan Open/Closed Principle (OCP) sebagai tambahan strategy baru tanpa mengubah sistem yang sudah ada.

10. Karyawan.java

```

1  public class Karyawan implements UktCalculateStrategy{
2
3      private static final double UKT_KARYAWAN = 80000000;
4
5      @Override
6      public double calculateUkt(DataUKT data) {
7          return UKT_KARYAWAN;
8      }
9  }
10

```

Karyawan.java memberikan tarif UKT khusus untuk mahasiswa karyawan dengan nilai tetap sebesar Rp 80.000.000 yang disimpan sebagai konstanta statis. Method

calculateUkt mengabaikan parameter DataUKT yang diterima dan langsung mengembalikan nilai konstan tersebut, menunjukkan bahwa kategori mahasiswa karyawan dikenakan tarif flat tanpa memperhatikan faktor jurusan, semester, atau prestasi. Prinsip SOLID yang diterapkan: Class ini menerapkan Single Responsibility Principle (SRP) dengan hanya menangani perhitungan UKT karyawan. Class ini juga menerapkan Liskov Substitution Principle (LSP) karena dapat menggantikan UktCalculateStrategy tanpa melanggar kontrak, dan Open/Closed Principle (OCP) sebagai bagian dari ekstensi strategy tanpa modifikasi context class.

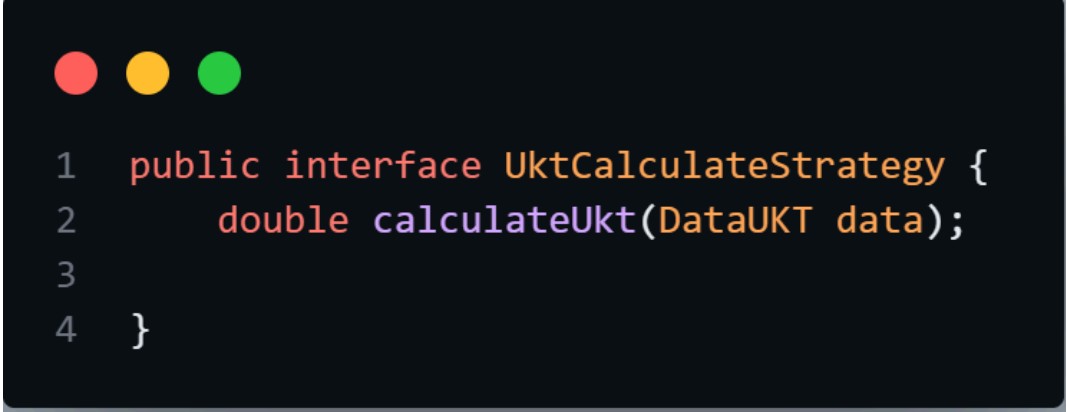
11. Regular.java

```
1 public class Regular implements UktCalculateStrategy {
2     private static final double UKT_DASAR = 50000000;
3     private static final double DISKON_BERPRESTASI = 20000000;
4     private static final String STATUS_BERPRESTASI = "berprestasi";
5
6     @Override
7     public double calculateUkt(DataUKT data) {
8         double uktDasar = UKT_DASAR;
9
10
11         if (data.getPrestasi().equalsIgnoreCase(STATUS_BERPRESTASI)) {
12             uktDasar -= DISKON_BERPRESTASI;
13         }
14
15         return uktDasar;
16     }
17 }
```

Regular.java mengimplementasikan perhitungan UKT untuk mahasiswa reguler dengan UKT dasar sebesar Rp 50.000.000 dan potongan biaya untuk mahasiswa berprestasi sebesar Rp 20.000.000. Method calculateUkt memulai dengan nilai dasar, kemudian memeriksa apakah atribut prestasi pada objek DataUKT mengandung string "berprestasi" menggunakan perbandingan case-insensitive, jika kondisi terpenuhi maka nilai diskon dikurangkan dari UKT dasar sehingga mahasiswa berprestasi hanya membayar Rp 30.000.000. Prinsip SOLID yang diterapkan: Class ini menerapkan Single Responsibility Principle (SRP) karena hanya bertanggung jawab untuk satu jenis perhitungan UKT. Class ini juga menerapkan Liskov Substitution Principle (LSP) karena dapat menggantikan

UktCalculateStrategy tanpa mengubah kebenaran program, dan Open/Closed Principle (OCP) karena dapat diperluas dengan inheritance tanpa mengubah kode yang ada.

12. UktCalculateStrategy.java



```
1 public interface UktCalculateStrategy {  
2     double calculateUkt(DataUKT data);  
3  
4 }
```

UktCalculateStrategy.java adalah interface yang mendefinisikan kontrak untuk pattern Strategy dalam perhitungan UKT, di mana setiap class konkret yang mengimplementasikannya harus menyediakan method `calculateUkt` yang menerima parameter `DataUKT` dan mengembalikan nilai `double` berupa jumlah biaya yang dihitung, sehingga memungkinkan pergantian algoritma perhitungan secara runtime. Prinsip SOLID yang diterapkan: Interface ini merupakan fondasi Open/Closed Principle (OCP) dan Dependency Inversion Principle (DIP) karena memungkinkan penambahan strategi perhitungan baru tanpa mengubah kode yang ada, serta membuat modul high-level (`CalculateUKT`) bergantung pada abstraksi ini, bukan pada detail implementasi.

13. MySQLDatabaseConnection



```
1  public interface MySQLDatabaseConnection {
2      void connect();
3      void disconnect();
4      boolean isConnected();
5      void executeQuery(String query);
6
7  }
8
```

MySQLDatabaseConnection.java adalah interface yang mendefinisikan kontrak untuk koneksi database MySQL dengan method connect untuk membuka koneksi, disconnect untuk menutup koneksi, isConnected untuk memeriksa status koneksi, dan executeQuery untuk menjalankan perintah SQL. Meskipun telah didefinisikan, interface ini belum diimplementasikan atau digunakan dalam alur program utama App.java, sehingga berstatus sebagai persiapan untuk pengembangan fitur database di masa depan. Prinsip SOLID yang diterapkan: Interface ini menerapkan Dependency Inversion Principle (DIP) dengan menyediakan abstraksi untuk koneksi database, sehingga class high-level nantinya akan bergantung pada interface ini, bukan pada implementasi database konkret. Interface ini juga menerapkan Interface Segregation Principle (ISP) dengan membatasi method-method yang esensial untuk koneksi database tanpa memaksakan method yang tidak relevan.

14. App.java

```

1  import java.util.ArrayList;
2  import java.util.List;
3  import KRS.*;
4  import MATKUL.*;
5  import UKT.*;
6
7  public class App {
8
9      public static void main(String[] args) {
10         System.out.println("=====");
11         System.out.println("    SISTEM AKADEMIK KAMPUS (SIKAD)    ");
12         System.out.println("=====\\n");
13
14         // 1. Daftar Mata Kuliah
15         System.out.println("--- [1] DAFTAR MATA KULIAH ---");
16
17         MataKuliah pbo = new MataKuliah() {
18             public String getNamaMatkul() {
19                 return "Pemrograman Berorientasi Objek";
20             }
21
22             public int getSKS() {
23                 return 3;
24             }
25         };
26
27         MataKuliah basisData = new MataKuliah() {
28             public String getNamaMatkul() {
29                 return "Basis Data";
30             }
31
32             public int getSKS() {
33                 return 3;
34             }
35         };
36
37         MatkulPraktikum prakPBO = new MatkulPraktikum("Praktikum PBO", 1);
38         MatkulPraktikum prakBasisData = new MatkulPraktikum("Praktikum Basis Data", 1);
39
40         System.out.println("Mata Kuliah Teori:");
41         System.out.println("  - " + pbo.getNamaMatkul() + " (" + pbo.getSKS() + " SKS)");
42         System.out.println("  - " + basisData.getNamaMatkul() + " (" + basisData.getSKS() + " SKS)");
43
44         System.out.println("\\nMata Kuliah Praktikum:");
45         System.out.println("  - " + prakPBO.getNamaMatkul() + " (" + prakPBO.getSKS() + " SKS)");
46         prakPBO.alokasiAsistenLab();
47         prakPBO.cekPeralatanPraktikum();
48
49         System.out.println("  - " + prakBasisData.getNamaMatkul() + " (" + prakBasisData.getSKS() + " SKS)");
50         prakBasisData.alokasiAsistenLab();
51         prakBasisData.cekPeralatanPraktikum();
52
53         // 2. Validasi Prasyarat
54         System.out.println("\\n--- [2] VALIDASI PRASYARAT MATA KULIAH ---");
55
56         ValidatorMatKul validator = new ValidatorMatKul();
57         String[] matkulDiuji = { "PBO", "Basis Data", "Algoritma", "Kecerdasan Buatan" };
58         String mahasiswaId = "MHS2024001";
59
60         for (String kode : matkulDiuji) {
61             boolean hasil = validator.validasiPrasyarat(mahasiswaId, kode);
62             System.out.println("  Hasil: " + (hasil ? "LULUS VALIDASI" : "GAGAL VALIDASI"));
63             System.out.println();
64         }
65
66         // 3. Cetak KRS
67         System.out.println("\\n--- [3] CETAK KRS MAHASISWA ---");
68
69         List<MataKuliah> daftarKRS = new ArrayList<>();
70         daftarKRS.add(pbo);
71         daftarKRS.add(basisData);
72         daftarKRS.add(prakPBO);
73         daftarKRS.add(prakBasisData);
74
75         KrsPrinter printer = new KrsPrinter();
76         printer.printPDF("Budi Santoso (NIM: MHS2024001)", daftarKRS);
77
78         // 4. Perhitungan UKT
79         System.out.println("\\n--- [4] PERHITUNGAN UKT ---");
80
81         DataUKT dataRegular = new DataUKT("Teknik Informatika", "Semester 3", "tidak berprestasi");
82         DataUKT dataRegularPrestasi = new DataUKT("Teknik Informatika", "Semester 5", "berprestasi");
83         DataUKT dataKaryawan = new DataUKT("Manajemen", "Semester 2", "tidak berprestasi");
84         DataUKT dataInternasional = new DataUKT("Kedokteran", "Semester 1", "tidak berprestasi");
85         DataUKT dataBidikMisi = new DataUKT("Teknik Sipil", "Semester 4", "berprestasi");
86
87         CalculateUKT calculator = new CalculateUKT();

```

Program dimulai dengan menampilkan header sistem, kemudian menginisialisasi dua mata kuliah teori yaitu Pemrograman Berorientasi Objek dan Basis Data menggunakan anonymous class yang mengimplementasikan interface MataKuliah, serta dua mata kuliah praktikum menggunakan class MatkulPraktikum. Setelah menampilkan daftar mata kuliah, program melakukan validasi prasyarat untuk empat mata kuliah yang berbeda dengan memanggil method validasiPrasyarat dari class ValidatorMatKul. Selanjutnya program mencetak Kartu Rencana Studi (KRS) untuk mahasiswa bernama Budi Santoso dengan mengumpulkan keempat mata kuliah ke dalam sebuah ArrayList dan meneruskannya ke method printPDF dari class KrsPrinter. Bagian inti program berada pada perhitungan Uang Kuliah Tunggal (UKT) di mana program membuat lima objek DataUKT dengan kombinasi jurusan, semester, dan status prestasi yang berbeda-beda, kemudian menggunakan class CalculateUKT sebagai context untuk mengganti-ganti strategy perhitungan secara dinamis mulai dari Regular, Regular dengan prestasi, Karyawan, Internasional, hingga Bidik Misi, di mana setiap perubahan strategy mengubah cara perhitungan biaya secara drastis. Program juga mendemonstrasikan penanganan error dengan sengaja membuat objek CalculateUKT tanpa strategy dan menangkap IllegalStateException yang terjadi. Di akhir program, ditampilkan ringkasan hasil perhitungan untuk memastikan semua fitur berjalan dengan benar. Prinsip SOLID yang diterapkan: App.java mengimplementasikan Dependency Inversion Principle (DIP) karena bergantung pada abstraksi berupa interface MataKuliah dan UktCalculateStrategy, bukan pada class konkret. App juga menerapkan Open/Closed Principle (OCP) secara tidak langsung karena dapat menambah kategori UKT baru tanpa mengubah kode di App.java, cukup dengan membuat class strategy baru.

VI. KESIMPULAN

Refactoring sistem akademik kampus dari monolitik menjadi berbasis prinsip SOLID berhasil mengatasi krisis arsitektur yang menyebabkan crash dan ketergantungan antar fitur. Pemisahan kelas SistemKRSManger menjadi entitas-entitas terpisah seperti KrsPrinter, ValidatorMatKul, dan CalculateUKT menerapkan Single Responsibility Principle (SRP), sehingga perubahan pada satu fitur tidak lagi merusak fitur lainnya. Penggunaan interface MataKuliah dan Praktikum secara terpisah menerapkan

Interface Segregation Principle (ISP) dan Liskov Substitution Principle (LSP), yang memungkinkan mata kuliah teori dan praktikum dikelola tanpa exception palsu. Implementasi Strategy Pattern melalui UktCalculateStrategy dengan kelas-kelas konkret Regular, Karyawan, Internasional, dan BidikMisi menerapkan Open/Closed Principle (OCP) dan Dependency Inversion Principle (DIP), memungkinkan penambahan jalur mahasiswa baru seperti MBKM tanpa merombak sistem yang ada. Interface MySQLDatabaseConnection yang belum diimplementasikan menyiapkan fondasi DIP untuk migrasi database ke Cloud NoSQL di masa depan. Secara keseluruhan, arsitektur baru ini lebih tangkas, tahan terhadap perubahan, dan siap mengakomodasi ekspansi fitur tanpa mengorbankan stabilitas sistem yang sudah berjalan.

VII. DAFTAR PUSTAKA

Tjondrojo, T.F. and Hutagalung, Y.S.C.I. (2026) Modul 12 - SOLID principles. Laboratorium Komputer, Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.